MDSec

MDSec
UNRIVALLED SECURITY CONSULTING AND TRAINING

ASSESSMENT REPORT

Gnosis Safe

Prepared for: cLabs

Prepared by: Marcus Pinto

marcus.pinto@mdsec.co.uk

+44 787 0484 157

Table of Contents

Table of Contents **2**
1. Document Control **3**
1.1 Document Details 3
1.2 Version History 3
2. Executive Summary **4**
2.1 Project Overview 4
2.2 Summary of Business Risks 4
2.3 Conclusions 4
3. Technical Information **5**
3.1 Assessment Scope 5
3.2 Caveats 5
3.3 Reading the CVSS3 Risk Rating 6
3.4 Assessment Results 7
4. Supporting Information **9**
4.1 Exposed Administrative Interface 9
4.2 Django debug mode enabled 10
5. Appendices **11**
5.1 Web Application Assessment Methodology 11

1. Document Control

This report contains confidential information and should not be disclosed outside of cLabs. This document is considered proprietary information and remains property of MDSec Consulting Limited.

1.1 Document Details

Client Name	cLabs
Author	Marcus Pinto marcus.pinto@mdsec.co.uk +44 787 0484 157
Protective Marking	Commercial In Confidence
Project Reference	Gnosis Safe
Document Reference	20211118_cLabs_Gnosis_Safe_RPT_v0.1
Document Version	1.0
Distribution List	Ryon Shamloo

1.2 Version History

Ver.	Date	Issue Description	Approval
0.1	27/01/2022	Initial Draft	Marcus Pinto
0.2	28/01/2022	Technical QA	Andrew McGregor
1.0	02/02/2022	Final Release to cLabs	Marcus Pinto

2. Executive Summary

2.1 Project Overview

MDSec performed an assessment of the "safe" feature implemented in Gnosis as a method of allowing multiple user access to crypto resources including cryptocurrency and NFTs.

The assessment covered the following areas:

- https://ui.celo-safe.io
- https://safe-transaction.gnosis.io/

The assessment was performed by MDSec consultant Marcus Pinto on 24/01/2021.

2.2 Summary of Business Risks

A total of 3 security weaknesses were identified. Of these, we consider 0 to be critical risk, 0 to be high risk, 2 to be medium risk, and 1 to be low risk. (<<vulnclosed>> resolved during the course of the assessment.) (3 issues have been reported for information purposes only.)

Risk	Issue Description	REF
Medium	**Exposed Administrative Interface** The Django administrative interface is enabled on safe-transaction.gnosis.io – attackers may attempt to brute force the administrator / superuser password in order to compromise the whole API. Related to this, the web server is running in debug mode, which leaks further information about resources on the server.	01, 03
Medium	**Expensive queries can be leveraged to cause denial of service** An expensive query was found which could be accessed by an attacker to cause denial of service, disrupting the API for all users.	02

2.3 Conclusions

Overall the application's primary function is to secure the integrity transaction process for safe contents and this is performed well. The application, acts as a minimal layer on top of the blockchain and wallet, which ensures that most core features such as transactions are securely implemented, whilst also leaving a minimal attack surface.

The application does not provide confidentiality; this is an expected feature of blockchain, however it is possible that users may assign safe names and safe owner names which identify individuals or which the users otherwise expect to be non-public.

In terms of availability, at least one API call was found which appears to result in excessive server side processing; an attacker wishing to bring down the service in question can do so with a single request issued periodically (every 2-3 seconds or so). It is possible that other such expensive calls exist; MDSec does not test for these deliberately in case such testing causes disruption for other users.

The Django instance found on safe-transaction.gnosis.io appears to be in a pre-production configuration, as it exposes the default administrative interface and is running in debug mode. This should be investigated and Django should be reconfigured for production.

3. Technical Information

3.1 Assessment Scope

The following systems formed the scope of assessment:

- https://ui.celo-safe.io
- https://transaction-service.celo-safe.io /

3.2 Caveats

No testing was performed with the intention of causing Denial of Service.

3.3 Reading the CVSS3 Risk Rating

MDSec rate vulnerabilities by CVSS3 score, an open standard used across many products and services to provide a uniform approach to vulnerability measurement. The CVSS3 base score is a carefully weighted score based on the Exploitability, Scope and Impact of an issue, and is banded into Critical, High, Medium and Low Risk categories according to the following ranges:

Critical	9.0 – 10.0
High	7.0 – 8.9
Medium	4.0 – 6.9
Low	0.1 – 3.9
None	0.0

These five risk ratings should be sufficient to understand the significance of a given issue. If further understanding is required, these overall ratings can be broken down into the following metrics (it should be noted MDSec only use Base metrics, while adding Environmental metrics when needed):

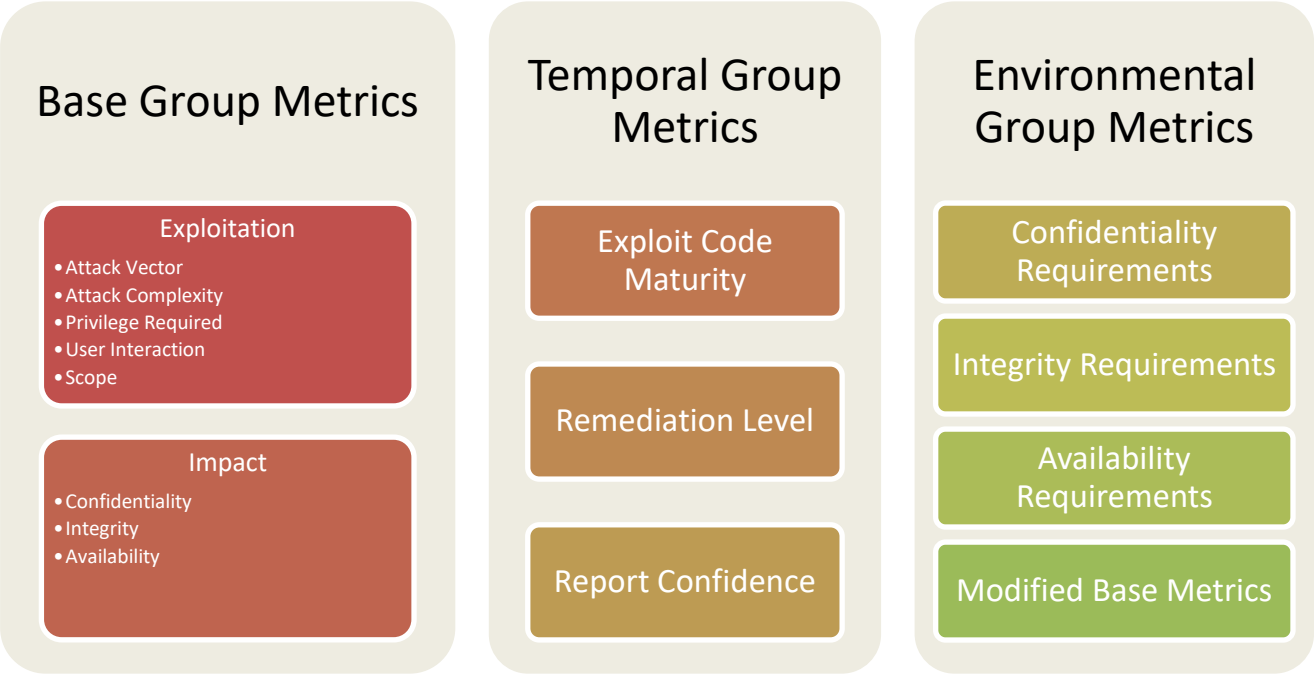


Figure 1 – CVSSv3 Score Metrics

Findings rated as "None" generally do not pose a direct threat to the security of the system; however they are reported on this report and marked as "INFO" to highlight deviation from best security practices and should ideally be addressed to provide defence in depth.

Full details on CVSS can be found at the following URLs:

- https://www.first.org/cvss/user-guide (CVSSv3 User Guide)
- https://nvd.nist.gov/vuln-metrics/cvss/v3-calculator (Reading a CVSS Vector)

3.4 Assessment Results

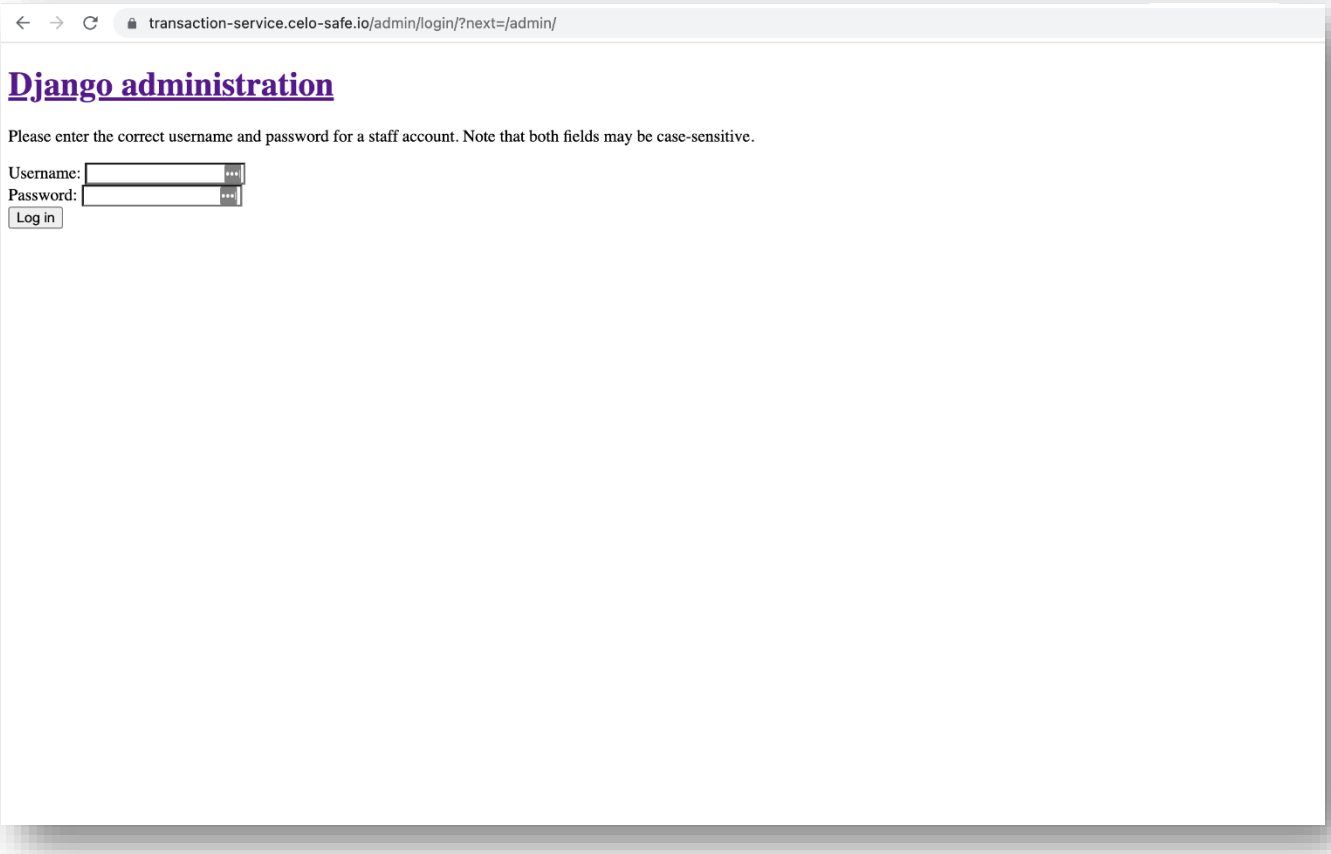
#	Vulnerability	CVSS	Description	Recommendation
01	**Exposed Administrative Interface** *transaction-service.celo-safe.io*	5.3	The application is developed in Django, and has the default Django administration interface available at the predictable resource location /admin. Whilst this does not pose an immediate threat to the application itself, it might aid an attacker in determining the name and possibly the version of the software in use, and offers the possibility of brute-force attacks against administrative user accounts. A screenshot can be found in Section 4.1.	MDSec recommends hiding access to the administrative interface. This could be applied by renaming the URL to a specific name that could not be guessed by common dictionary attacks. This could also be managed by enabling access controls based on the user who is trying to access the interface. This could be achieved by having a whitelist of allowed IP addresses.
02	**Expensive queries causing denial of service** *transaction-service.celo-safe.io*	5.1	During the assessment it was discovered that it is possible to make an expensive API call which may result in temporary denial of service (or permanent Denial of Service if the request is repeated). The `/analytics/multisig-transactions-by-safe` function allows multiple safes to be viewed in one response. Specifying all 33586 transactions at the time of testing resulted in an HTTP 503 Service Unavailable HTTP error for a brief period. This was not retested as there was no desire to deliberately cause denial of service, but it is likely that repeat requests to this API would result in service unavailability. https://safe-transaction.gnosis.io/api/v1/analytics/multisig-transactions/by-safe/?limit=33586&offset=1	Consider setting an upper limit on the number of safes that can be viewed at a time in this manner.
03	**Django debug mode enabled** *transaction-service.celo-safe.io*	3.4	The application is built using the popular python framework Django. This is currently running in its "debug" mode; in this mode error messages leak implementation details including the routes supported by the application. See Section 4.2 for an example.	Debugging information is as useful to an attacker as it is to the intended recipient. Disable debugging in production applications.

#	Vulnerability	CVSS	Description	Recommendation
04	**Cacheable HTTPS Response** *ui.celo-safe.io*	INFO	Unless directed otherwise, browsers may store a local cached copy of content received from web servers. Some browsers, including Internet Explorer, cache content accessed via HTTPS. If sensitive information in application responses is stored in the local cache, then this may be retrieved by other users who have access to the same machine at a future time. The application does not explicitly prevent caching, and consequently sensitive data may be cached. This issue is listed only as informational, as it is understood that data is not expected to be private, however caching data such as this may allow someone to connect safes with individuals.	The application should return caching directives instructing browsers not to store local copies of any sensitive data. Often, this can be achieved by configuring the web server to prevent caching for relevant paths within the web root. Alternatively, most web development platforms allow you to control the server's caching directives from within individual scripts. Ideally, the web server should return the following HTTP headers in all responses containing sensitive content: Cache-control: no-store Pragma: no-cache
05	**Publicly accessible safe information** *transaction-service.celo-safe.io*	INFO	It was noted that the APIs on transaction-service.celo-safe.io provide a significant quantity of information on safes. Some of this was expected by MDSec as it includes public information on all transactions, however some was less expected, namely the ability to list all users' safe names and safe contents. As there is nothing indicating that this information should be expected to be public, it may be a surprise to some users.	During discussions with cLabs it was confirmed that this is expected behaviour and that users' safe names and contents should be public knowledge as part of the overall ethos of cryptocurrency transparency. MDSec have reported this issue as informational only as a result but suggest that users are made aware of this.
06	**Web Service Discloses Software Type and Version** *transaction-service.celo-safe.io*	INFO	The web service discloses its software type and version in the HTTP response to requests. By using this information, it is possible to fingerprint the web server in use as well as the underlying operating system. Although this should not be considered a vulnerability in itself, it does provide an attacker with sufficient information to tailor more targeted attacks against the web server. Submitting a TRACE request to the root directory returned an "`HTTP/2 405 Method Not Allowed`" error message identifying `nginx` as the server.	Where possible, it is recommended that default software banners be changed to restrict the disclosure of version information. For Nginx, the following line should be added to the `http` configuration context in the `nginx.conf` file: server_tokens off;

4. Supporting Information

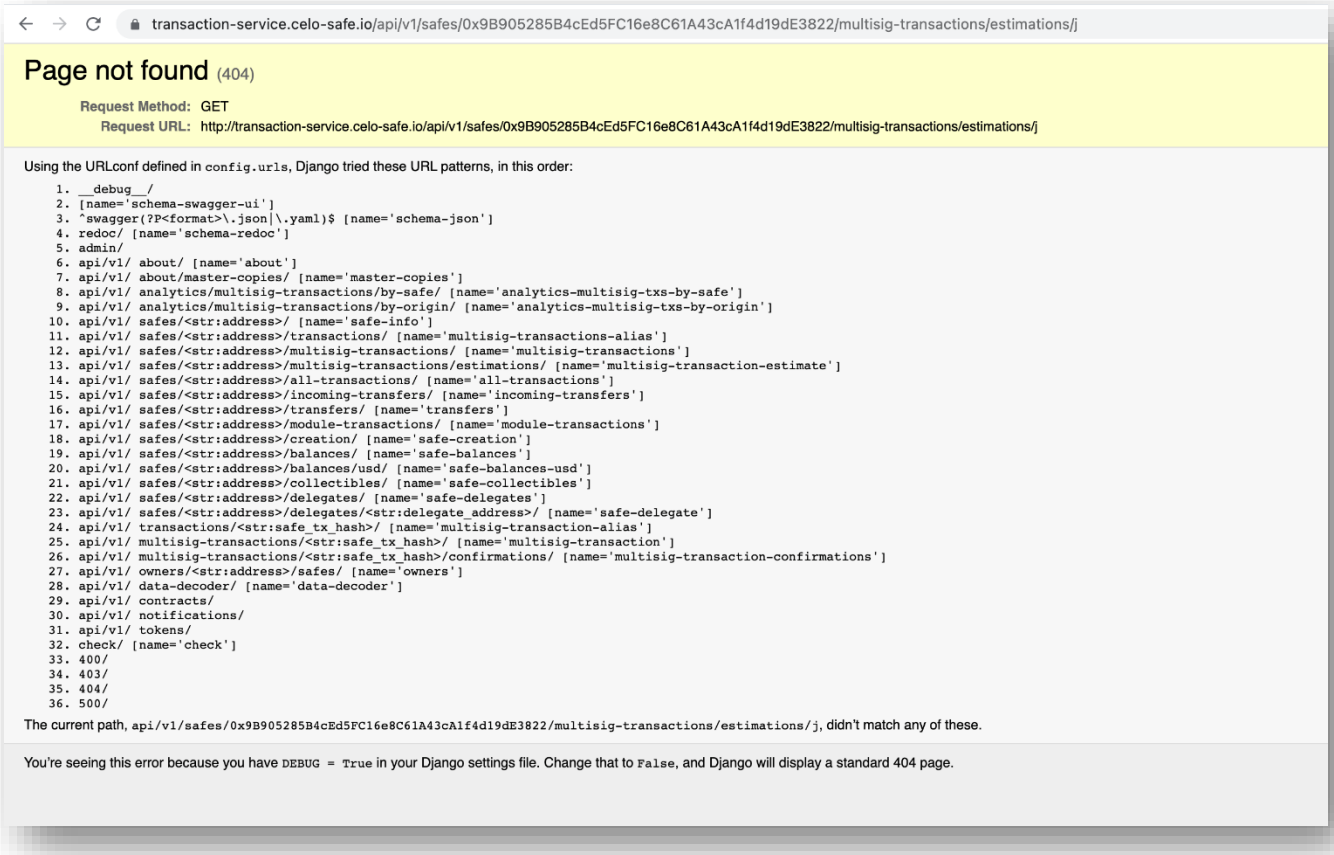
4.1 Exposed Administrative Interface

The following administration interface is accessible from any internet-facing IP address:



4.2 Django debug mode enabled

As Django's debug mode is enabled in the application, it is possible to see debugging errors which inform the user about all of the available routes (URLs) in the application:



transaction-service.celo-safe.io/api/v1/safes/0x9B905285B4cEd5FC16e8C61A43cA1f4d19dE3822/multisig-transactions/estimations/j

Page not found (404)

Request Method: GET
Request URL: http://transaction-service.celo-safe.io/api/v1/safes/0x9B905285B4cEd5FC16e8C61A43cA1f4d19dE3822/multisig-transactions/estimations/j

Using the URLconf defined in `config.urls`, Django tried these URL patterns, in this order:

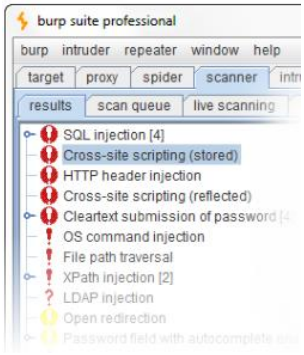
```
1. __debug__/
2. [name='schema-swagger-ui']
3. ^swagger(?P<format>\.json|\.yaml)$ [name='schema-json']
4. redoc/ [name='schema-redoc']
5. admin/
6. api/v1/ about/ [name='about']
7. api/v1/ about/master-copies/ [name='master-copies']
8. api/v1/ analytics/multisig-transactions/by-safe/ [name='analytics-multisig-txs-by-safe']
9. api/v1/ analytics/multisig-transactions/by-origin/ [name='analytics-multisig-txs-by-origin']
10. api/v1/ safes/<str:address>/ [name='safe-info']
11. api/v1/ safes/<str:address>/transactions/ [name='multisig-transactions-alias']
12. api/v1/ safes/<str:address>/multisig-transactions/ [name='multisig-transactions']
13. api/v1/ safes/<str:address>/multisig-transactions/estimations/ [name='multisig-transaction-estimate']
14. api/v1/ safes/<str:address>/all-transactions/ [name='all-transactions']
15. api/v1/ safes/<str:address>/incoming-transfers/ [name='incoming-transfers']
16. api/v1/ safes/<str:address>/transfers/ [name='transfers']
17. api/v1/ safes/<str:address>/module-transactions/ [name='module-transactions']
18. api/v1/ safes/<str:address>/creation/ [name='safe-creation']
19. api/v1/ safes/<str:address>/balances/ [name='safe-balances']
20. api/v1/ safes/<str:address>/balances/usd/ [name='safe-balances-usd']
21. api/v1/ safes/<str:address>/collectibles/ [name='safe-collectibles']
22. api/v1/ safes/<str:address>/delegates/ [name='safe-delegates']
23. api/v1/ safes/<str:address>/delegates/<str:delegate_address>/ [name='safe-delegate']
24. api/v1/ transactions/<str:safe_tx_hash>/ [name='multisig-transaction-alias']
25. api/v1/ multisig-transactions/<str:safe_tx_hash>/ [name='multisig-transaction']
26. api/v1/ multisig-transactions/<str:safe_tx_hash>/confirmations/ [name='multisig-transaction-confirmations']
27. api/v1/ owners/<str:address>/safes/ [name='owners']
28. api/v1/ data-decoder/ [name='data-decoder']
29. api/v1/ contracts/
30. api/v1/ notifications/
31. api/v1/ tokens/
32. check/ [name='check']
33. 400/
34. 403/
35. 404/
36. 500/
```

The current path, `api/v1/safes/0x9B905285B4cEd5FC16e8C61A43cA1f4d19dE3822/multisig-transactions/estimations/j`, didn't match any of these.

You're seeing this error because you have `DEBUG = True` in your Django settings file. Change that to `False`, and Django will display a standard 404 page.

5. Appendices

5.1 Web Application Assessment Methodology



MDSec intercept network traffic between the application and the server, using our Burp Suite proxy tool to modify requests and responses, and inject custom faults into the system to test for common attacks such as Client-side Access Controls, Logic flaws, session handling, replay attacks and other logic flaws, as well as using a combination of manual and automated techniques to assess the application for input-based vulnerabilities such as XSS, SQL Injection (client and server side), File path manipulation, and custom language injection. Using Burp Extender interfaces and a combination of Ruby, Java and .NET code we are able to adapt our attack platform to best effect against your application, de-serialising data, running custom decoders, and performing in-session detection.

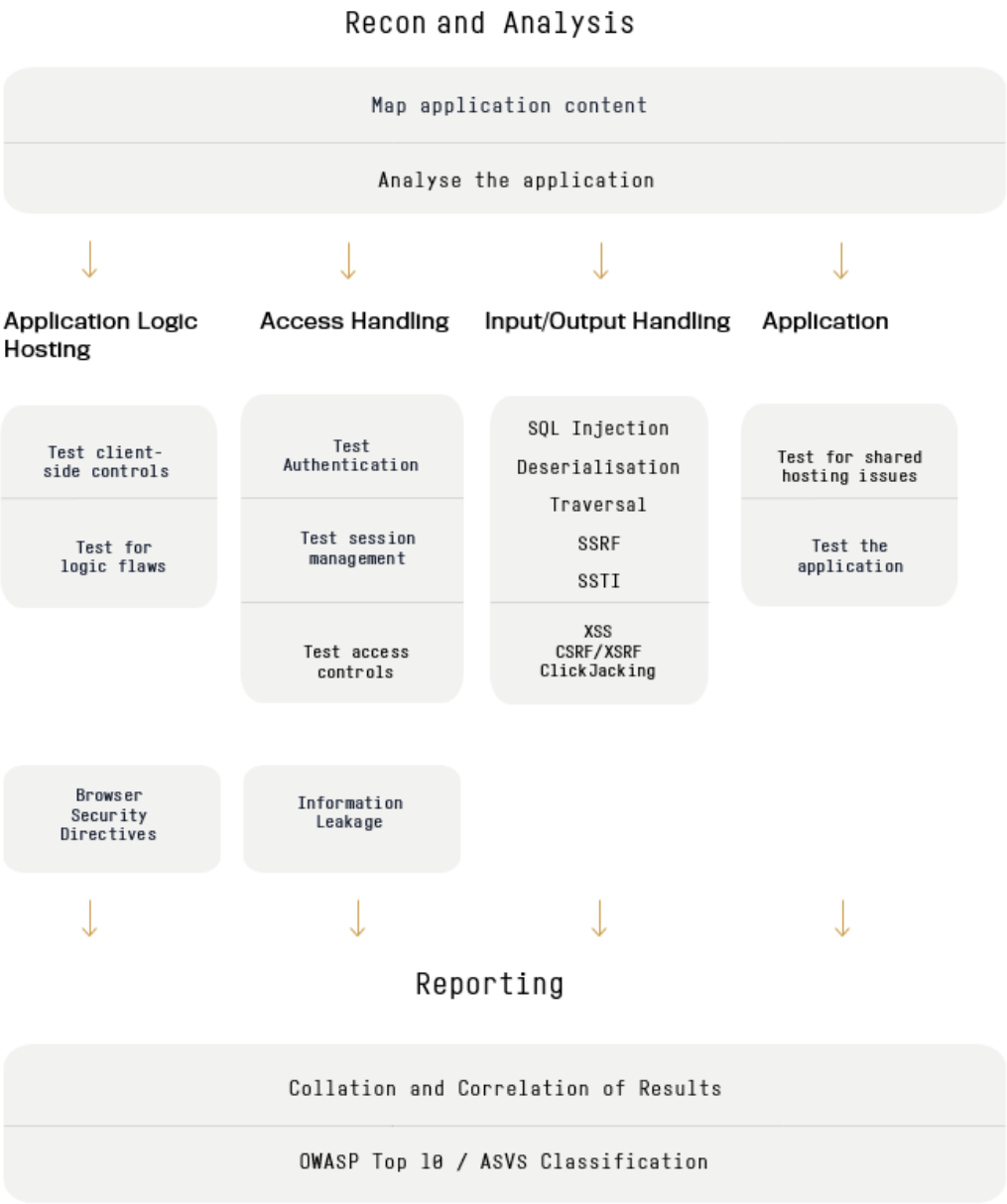


Figure 2 - The assessment process for Web Application Assessment.